

AULA 07

Quem controla o tempo do jogo

2026.2

Roteiro da noite

- 1 Por que três?
as razões para separar inputs, update e draw
- 2 A IA digita
especificamos, ela refatora, nós conferimos
- 3 Os quatro tipos
as formas de controlar a velocidade
- 4 Onde estamos
e como descer e subir um degrau
- 5 O engasgo
o que o passo variável quebra

Onde a gente parou

O Pong da IA: um arquivo, nenhuma classe.

A bola gruda na raquete.

E uma linha decorativa: `clock.tick(60)`.

Hoje a linha decorativa vira o assunto.

O jogo funciona num bloco só. Por que separar?

E não vale “porque fica mais organizado”. Isso não é uma razão.

Um quadro que nunca existiu

```
bola.mover()  
desenhar(bola)      # dentro da raquete  
resolver_colisao()  # só agora tira ela de lá
```

O jogador viu um estado que o jogo nunca teve.

Pausar é parar de chamar update

```
if not pausado:  
    update()  
draw()                # continua desenhando
```

Num bloco só, não há onde enfiar esse if.

Câmera lenta e replay saem daqui também.

Rodar o jogo sem abrir o jogo

O bug pode ser achado sem jogar.

Rodar várias vezes, sem janela.

Só é possível se o update não precisar do draw.

Separar não é arrumação. É o que torna o jogo testável.

Atualizar mais que desenhar

```
update()  
update()      # o mundo andou duas vezes  
draw()        # a tela mostrou uma
```

Máquina rápida: desenha mais que atualiza.

Máquina lenta: atualiza mais que desenha.

Num bloco só, é sempre a mesma taxa.

Especificar antes de digitar

```
"Separe este laço em tres funcoes:  
inputs(), update() e draw().  
Não mude nenhum comportamento."
```

Sobrou desenho dentro do update?

Roda igual — inclusive o bug?

Quatro formas de controlar a velocidade

- 1 Passo fixo, sem sincronização
a velocidade é a da máquina
- 2 Passo fixo, com sincronização ← **estamos aqui**
espera o que falta do quadro
- 3 Passo variável ← **hoje, veremos**
o passo é o tempo que passou
- 4 Update fixo + render variável
física travada, desenho livre

Nós estamos no tipo 2

```
clock.tick(60)
```

Ele espera. É o que impede o jogo de acelerar.

Ninguém aqui escolheu o tipo 2. A IA escolheu.

Medir e descontar

```
inicio = agora()  
inputs()  update()  draw()      # levou 5 ms  
esperar  16,7 - 5   =  11,7 ms
```

É isso que o `clock.tick(60)` faz por você.

Apague a linha e rode

5 px por quadro × o que a máquina der.

Isso é o tipo 1.

Por isso jogos de DOS ficaram injogáveis.

Ele protege de um lado só

trabalho 5 ms + espera 11,7 ms = 16,7 ms

trabalho 50 ms + espera 0 = 50,0 ms

Máquina rápida: sobra tempo, e ele espera.

Máquina lenta: não existe espera negativa.

O jogo fica três vezes mais lento.

O passo vira o tempo que passou

```
dt = clock.tick(60) / 1000    # ms para s  
bola.x += 300 * dt
```

5 px por quadro

× 60 quadros por segundo

= **300 px por segundo**

O 7 da raquete vira 420. E assim por diante.

300 px por segundo, sempre

60 fps $dt = 0,0167 \text{ s}$ passo = 5 px

30 fps $dt = 0,0333 \text{ s}$ passo = 10 px

Metade dos quadros, o dobro do passo.

Em um segundo, a bola andou 300 px nas duas.

Agora estamos no tipo 3

```
while rodando:  
    dt = clock.tick(60) / 1000  # ms para s  
    inputs(dt)  
    update(dt)  
    draw()
```

AULA 07

Como controlar o tempo do jogo

2026.2